COSC 4370 – Homework 3

Name: Samuel Heng PSID: 1580496

September 2023

1 Problem

This assignment requires the implementation of 3D viewing and Phong shading model to shade a simple cube. The Phong shading algorithm is defined as $L = L_a + L_d + L_s$ which is the sum of the three key components, where $L_a = ambient\ lighting, L_d = diffuse\ lighting, and\ L_s = specular\ lighting.$

2 Method

There are four files that needed to be modified in the provided codes: *main.cpp, Camera.h, phong.vs, and phong.frag*. In order from least amount of modifications, to most: *main.cpp* sets up the cube's geometry and projection matrix to ensure the correct perspective; *Camera.h* creates a view matrix that transforms world-space coordinates to camera-space coordinates; *phong.vs* is the vertex shader which passes transformed vertex positions and normals to the fragment shader, and *phong.frag* is the fragment shader which calculates the Phong shading algorithm for each fragment to produce the final shaded object.

The general idea behind this method was to interpolate normal vectors and compute the full shading model across each pixel using the Phong shading algorithm. Although this method requires more computation, it allows for better shading of smooth surfaces.

3 Implementation

Given the provided files to modify, main.cpp, Camera.h, phong.vs, and phong.frag; the projection matrix, camera position and direction are calculated to ensure that the cube is correctly proportional to the viewing window, and easy to navigate. The vertex shader and fragment shader work in unison to calculate the correct fragment position and compute each component of the Phong shading algorithm to return each final pixel colors.

3.1 main.cpp

The main purpose of this file is the rendering of a Phong shaded cube in an interactive 3D window. This includes defining the

cube's geometry, using vertex and fragment shaders to render the final cube, handling user input for camera movement, and most importantly, passing view and projection matrices to the shader.

Before rendering the final cube, the project matrix must be set up by setting projection to:

$$glm::perspective(fov, aspectRatio, 0.1f, 100.0f).$$

glm::perspective is a function that is used to create the projection matrix by converting vectors from eye space to clip space. This is implemented by calculating the *aspectRatio* to ensure that the cube is correctly proportioned to the viewing window, *FOV(field-of-view)* which is the angle that defines the width of the cameras viewing area, and the *near* and *far clipping planes* which determines the rendering space in the camera's view. Since Phong shading requires more computation than other shading methods, it is important to set the correct *aspectRatio, FOV*, and *clipping planes* to ensure accurate and efficient rendering.

3.2 Camera.h

The camera class is an essential component to this assignment because it handles the camera's position and direction. The *GetViewMatrix()* function is responsible for generating and returning the view matrix. This is implemented by calling the GLM function:

$$glm::lookAt(Position, Position + Front, Up);$$

glm::lookAt() creates the view matrix using: '*Position*' which is the camera's position in the world space, '*Position + Front*' which defines the camera's viewing direction, and '*Up*' which is the upward direction vector that determines the camera's orientation and rotation point. Overall, the view matrix allows for the transformation of objects from world space to camera space. This file specifically handles user keyboard and mouse input to control movement and orientation, enabling viewers to easily navigate the camera window.

3.3 phong.vs

This is the vertex shader which is responsible for implementing the Phong shading model. In this application we have the input attributes: '*layout(location = 0) in vec3 position'* and '*layout(location = 1) in vec3 normal'*, which represent the position and normal vector of each vertex. These attributes are transformed

into the outputs: '*out vec3 Normal*' and '*out vec3 FragPos'*, which represent the interpolated fragment position and normal for each pixel on the cube's surface

The implementation starts with calculating the fragment position:

$$FragPos = vec3(\ model * vec4(position, 1.0)).$$

Here we are converting the local coordinates of the model into a 3D vector that represents the position of the vertex in world space. This is done by calculating $vec3(model * vec4(position, 1.0))$, where *'model'* which is the model matrix that transforms vertices to world space, *'vec4(position, 1.0)'* which is the 4D vector created with 3D position of the vertex, *'position,'* and *'1.0'* which is a homogenous coordinate (*w = 1)* allowing for uniform transformations. Next is calculating the vertex normal, '*Normal'*:

$$Normal = mat3\left(transpose\left(inverse(model)\right)\right) * normal.$$

Firstly, '*inverse(model)*' ensures that the correct normal vector is returned since the model matrix may include transformations that can affect the normal vector direction. '*transpose(inverse(model))'* returns the normal matrix, ensuring correct scaling. Lastly, we calculate the product of the normal matrix and vertex normal to transform the object space into world space.

Lastly, the final position of the vertex in clip space '*gl_Position'* is computed by:

$$projection * view * (model * vec4(position, 1.0))$$

One component of calculating the final vertex position is like the calculation of fragment position, where we have:

'*(model * vec4(position, 1.0))*' which applies the model matrix to the vertex in object space and converting it to world space. Next we multiply '*view'* which is the camera view matrix that transforms the vertices from world space to camera space. Furthermore, to bring the vertices from camera space to clip space, we must apply the projection matrix by multiplying '*projection'* to '*view * (model * vec4(position, 1.0))'*.

3.4 phong.frag

To finish the Phong shading model, this fragment shader is used to calculate the final fragment color and produce the realistic lighting

model for the cube. This is calculated by the Phong shading algorithm:

$$L = L_{ambient} + L_{diffuse} + L_{specular}.$$

The fragment shader starts with the function '*phongShading()*' which calculates the ambient, diffuse, and specular components, and returns the calculated Phong shading model. The first component, $vec3\ ambient = 0.1f * lightColor$; calculates the ambient lighting which is a uniform, and non-directional light in an environment:

$$L_a = K_a * L_{Color} \text{ where,}$$

$$L_a = ambient\ light\ K_a = ambient\ reflection\ coefficient$$

$$L_{Color} = light\ color$$

The second component diffuse lighting:

$$vec3\ diffuse = \max(dot(Normal, lightDirection), 0.0) * lightColor;$$

Calculates how the light scatters or diffuses on the surface of the cube. This is computed by:

$$L_d = K_d * L_{color} * \max\ (dot(N, L), 0)$$

$$L_d = diffuse\ light\ K_d = reflection\ coefficent$$

$$L_{Color} = light\ color$$

Diffuse lighting depends on the angle between the surface normal and the light direction, $\max(dot(Normal, lightDirection), 0.0)$, where we take the max value of the dot product of surface normal, and light direction. The max value is returned in the case that the diffuse variable returns negative.

Lastly, the third component specular lighting:

$$vec3\ specular = pow(\max(dot(viewDirection, lightDirection), 0.0)\ , 64.0) * 0.5 * lightColor$$

Is responsible for the highlights on the cube's surface. This is computed by:

$$L_s = K_s * L_{Color} * \max(dot(R, V), 0)^n$$

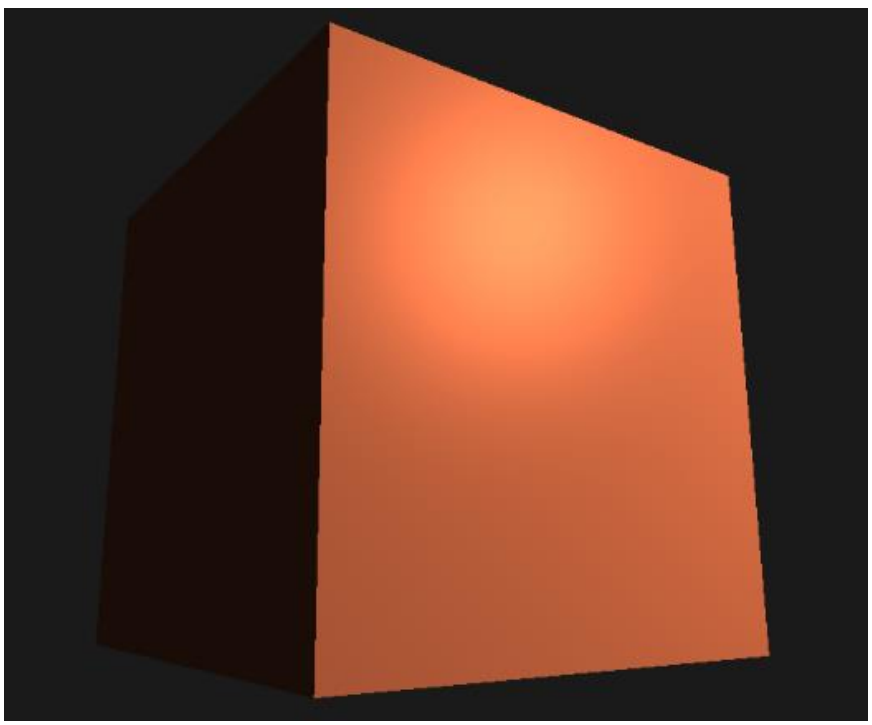
$$L_s = specular\ lighting\ K_s = reflection\ coefficient$$

$$L_{Color} = light\ color$$

Specular lighting depends on the angle between the viewing direction and the reflection of the light direction, $\max(dot(R, V), 0)^n$, where we compute the dot product of the normalized reflection vector, and the normalized viewing direction vector. The max is raised to the power n which controls the size and sharpness of the specular highlights. With all three components combined: $Ambient + Diffuse + Specular$, the final pixel color is returned.

4 Result

The resulting output is an interactive viewing window, controlled by the user's mouse and keyboard input. The viewing window features a red, 3D rendered cube that is shaded using the Phong shading model which provides a visually appealing cube with depth and dimension.



Sources used:

- Understanding GLM's Perspective Function
- Basic Lighting by LearnOpenGL
- Phong Reflection Model Wikipedia